

ANÁLISIS COMPARATIVO DE LAS TECNOLOGÍAS ESP32 Y RASPBERRY PI PICO PARA SISTEMAS EMBEBIDOS

Vásquez Menjívar, Luis Enrique.

Facultad de Ingeniería y Arquitectura, Universidad de El Salvador

Notas del Autor

Carnet: VM15037

Grupo: 02

No tenemos ningún conflicto de intereses que revelar.

La correspondencia relativa a este artículo deberá dirigirse al correo: vm15037@ues.edu.sv

Resumen

El presente estudio desarrolla un análisis técnico comparativo entre el microcontrolador ESP32 clásico de Espressif Systems y el microcontrolador RP2040 de la Raspberry Pi Foundation, con el propósito de evaluar sus capacidades, arquitecturas y aplicación en sistemas embebidos e Internet de las Cosas (IoT).

La metodología se basa en la revisión de documentación técnica oficial y literatura especializada, complementada con la implementación de prototipos simulados en entornos como Wokwi. El análisis considera variables como arquitectura de procesamiento, memoria, periféricos, conectividad, consumo energético y mecanismos de seguridad.

Los resultados evidencian que el ESP32, basado en una arquitectura Xtensa LX6 de doble núcleo a alta frecuencia, destaca por su alto nivel de integración, incorporando conectividad inalámbrica (Wi-Fi y Bluetooth) y soporte para sistemas operativos en tiempo real, lo que lo posiciona como una solución eficiente para aplicaciones IoT distribuidas. Por su parte, el Raspberry Pi, basado en núcleos ARM Cortex-M0, presenta mejoras significativas en eficiencia, seguridad y control de hardware, incluyendo mecanismos avanzados como TrustZone y subsistemas especializados de entrada/salida programable que permiten un comportamiento determinista.

Las pruebas prácticas demuestran que ambas plataformas son capaces de implementar soluciones de monitoreo con sensores; sin embargo, presentan ventajas diferenciadas según el contexto de aplicación; El ESP32 resulta más adecuado en escenarios que requieren conectividad integrada y mayor capacidad de procesamiento, mientras que el Raspberry Pi destaca en sistemas embebidos que demandan precisión temporal, seguridad y eficiencia energética.

Palabras claves: ESP32, Raspberry Pi Pico, microcontroladores, sistemas embebidos, IoT.

Introducción

El diseño de sistemas embebidos modernos exige la evaluación sistemática de las plataformas de hardware disponibles, con el fin de seleccionar aquella que mejor se adapte a los requerimientos funcionales y no funcionales del proyecto. En la actualidad, el mercado ofrece una amplia variedad de microcontroladores y sistemas en chip (SoC) que compiten en términos de capacidad de procesamiento, consumo energético, conjunto de periféricos y ecosistema de desarrollo. Entre estas opciones, el ESP32 desarrollado por Espressif Systems y lanzado en 2016, se ha establecido como el estándar de facto para dispositivos conectados gracias a su integración de Wi-Fi y Bluetooth en un solo chip. Este sistema en chip (SoC) utiliza la arquitectura Xtensa LX6, diseñada para ofrecer eficiencia en el procesamiento de tareas de red y aplicaciones generales.

Por otro lado, tenemos la Raspberry Pi Pico de la Fundación Raspberry Pi que ingresó al mercado en 2021 con el microcontrolador RP2040, introduciendo una filosofía de diseño centrada en la flexibilidad de los periféricos y el determinismo mediante su subsistema de Entrada/Salida Programable (PIO). Actualmente ambas alternativas son ampliamente utilizadas por la comunidad de desarrolladores, investigadores y educadores.

La comparación es relevante porque el mercado de microcontroladores ya no se organiza solo por frecuencia de reloj, sino por el equilibrio entre conectividad, periféricos, consumo, seguridad y ecosistema de software. En ese sentido, el análisis técnico de estas plataformas permite identificar no solo diferencias de especificaciones, sino también diferencias de enfoque arquitectónico y de campo de aplicación.

Objetivos

General

- Realizar un análisis comparativo entre el microcontrolador ESP32 clásico de Espressif Systems y el microcontrolador RP2040 de la Raspberry Pi Pico, evaluando sus arquitecturas, capacidades técnicas y desempeño en aplicaciones de sistemas embebidos e Internet de las Cosas (IoT).

Específicos

- Describir la arquitectura, especificaciones técnicas, periféricos y ecosistemas de desarrollo del ESP32 y la Raspberry Pi Pico.
- Comparar las características técnicas de ambas plataformas (frecuencia, memoria, GPIO, conectividad, entre otros) mediante una tabla comparativa, identificando similitudes y diferencias.
- Analizar las ventajas y desventajas de cada plataforma en función de su rendimiento y capacidades técnicas.
- Implementar y evaluar un proyecto simulado en Wokwi en ambas plataformas, que incluya la lectura de al menos dos sensores y la visualización de datos, analizando el comportamiento, desempeño y facilidad de desarrollo en cada caso.
- Establecer criterios de selección tecnológica según escenarios de aplicación, tales como IoT inalámbrico, sistemas embebidos deterministas y soluciones de bajo consumo.

Arquitectura de procesamiento y organización del silicio

ESP32: Arquitectura, CPU, memoria, conectividad y periféricos

El ESP32 es un SoC de propósito general orientado a soluciones conectadas. Se basa en el microprocesador Tensilica Xtensa LX6 de 32 bits, disponible en versiones de uno o dos núcleos (PRO_CPU y APP_CPU). Esta arquitectura destaca por su capacidad de operar hasta a 240 MHz, ofreciendo un rendimiento excepcional para tareas computacionalmente intensivas. Una característica técnica distintiva de la arquitectura Xtensa es su archivo de registros por ventanas (Windowed Register File). Mientras que la mayoría de las arquitecturas RISC disponen de un conjunto fijo de registros visibles, el LX6 posee 64 registros físicos de 32 bits, pero solo permite que 16 de ellos sean visibles en un momento dado mediante un registro especial llamado WindowBase. Este mecanismo optimiza las llamadas a funciones al desplazar la ventana de registros en lugar de guardar y restaurar datos de la pila en memoria, lo que reduce significativamente la sobrecarga en el cambio de contexto y la ejecución de subrutinas complejas.

Posee un chip combinado de Wi-Fi y Bluetooth de 2,4 GHz fabricado en tecnología de 40 nm, con una gestión de memoria exhaustiva ya que el mapa de direcciones de 4 GB permite el acceso simétrico a memorias embebidas y periféricos. La SRAM interna está dividida en tres bloques: SRAM0 (192 KB para instrucciones), SRAM1 (128 KB para datos) y SRAM2 (200 KB para datos DMA). Además, el ESP32 soporta hasta 16 MB de memoria Flash SPI externa y PSRAM adicional a través de una caché transparente, lo que permite manejar aplicaciones con grandes pilas de red o interfaces gráficas. También integra aceleradores criptográficos, co-procesador de ultra bajo consumo y un conjunto amplio de periféricos que incluye, entre otros, 34 pines GPIO programables, convertidores analógico-digiales (ADC) de 12 bits con hasta 18 canales, convertidores digital-analógico (DAC) de 8 bits, interfaces UART, SPI, I²C e I²S, controladores PWM para control de motores y LEDs,

un sensor de temperatura integrado, sensores táctiles capacitivos, y un controlador de bus CAN (Espressif Systems, 2024). Esta riqueza de periféricos reduce la necesidad de componentes externos y facilita la integración de sensores y actuadores.

Desde el punto de vista funcional, el ESP32 fue concebido para aplicaciones donde la conectividad inalámbrica es parte del núcleo del sistema. Espressif lo describe como una solución muy integrada para Wi-Fi y Bluetooth, con soporte para protocolos 802.11 b/g/n y Bluetooth v4.2 BR/EDR y BLE; además, su modelo de consumo contempla estados como light-sleep y deep-sleep, lo cual lo hace útil en dispositivos móviles, wearables e IoT. La documentación de FreeRTOS en ESP-IDF confirma que el sistema utiliza una implementación SMP adaptada a los chips de doble núcleo, lo que refuerza su orientación a multitarea concurrente.

En términos de evolución tecnológica, la familia ESP32 se expandió más allá del modelo clásico hacia distintas series soportadas por ESP-IDF, entre ellas ESP32-S, ESP32-C, ESP32-H y ESP32-P. Esto demuestra que el ecosistema no debe entenderse como un único chip aislado, sino como una plataforma que evolucionó hacia variantes con distintos compromisos entre rendimiento, radio, costo y consumo.

Raspberry Pi Pico: Arquitectura, CPU, memoria, periféricos y PIO

Por su parte, el RP2040 implementa dos núcleos ARM Cortex-M0+ basados en la arquitectura ARMv6-M. A diferencia del enfoque de rendimiento bruto del ESP32, el Cortex-M0+ está diseñado para la eficiencia energética y el bajo conteo de puertas lógicas. Utiliza un pipeline de solo dos etapas, lo que minimiza el consumo de energía y permite un determinismo temporal muy elevado, ideal para aplicaciones donde el tiempo de ejecución

debe ser constante. El RP2040 opera a una frecuencia nominal de 133 MHz, admite overclocking por encima de los 400 MHz

Este chip, fabricado en un proceso de 40 nm, representa la incursión de la fundación en el ámbito de los microcontroladores, complementando su línea de computadoras de placa única. A diferencia del ESP32, el RP2040 es un microcontrolador puro: carece de conectividad inalámbrica integrada y de memoria flash interna. En lo que respecta a la memoria, el RP2040 integra 264 KB de SRAM organizada en seis bancos independientes, lo que permite accesos simultáneos desde diferentes periféricos y núcleos sin contención. La placa Raspberry Pi Pico incluye, además, 2 MB de memoria flash externa conectada mediante una interfaz QSPI. Es importante señalar que el RP2040 no posee ROM ni flash interna; el firmware se carga desde la memoria flash externa o a través del bus USB durante el arranque. Esta arquitectura, aunque diferente de la del ESP32, proporciona una separación clara entre el almacenamiento no volátil y la memoria de trabajo.

El subsistema de periféricos del RP2040 incluye 30 pines GPIO, de los cuales 4 pueden configurarse como entradas analógicas para el ADC de 12 bits. Asimismo, el chip dispone de 2 controladores UART, 2 controladores SPI, 2 controladores I²C, 16 canales PWM y un controlador USB 1.1 (Raspberry Pi Foundation, 2021). Estos periféricos estándar cubren la mayoría de las necesidades de comunicación y control en proyectos embebidos convencionales.

La característica más innovadora del RP2040 es la unidad de Entrada/Salida Programable (PIO). Se trata de dos bloques PIO, cada uno con cuatro máquinas de estado independientes, lo que suma un total de ocho máquinas de estado programables (Raspberry Pi Foundation, 2021). Cada máquina de estado puede ejecutar un programa compuesto por hasta 32 instrucciones de un conjunto específico de nueve operaciones: JMP, WAIT, IN, OUT, PUSH, PULL, MOV, IRQ y SET (Raspberry Pi Foundation, 2021). Estas máquinas de estado

operan de manera independiente de los núcleos Cortex-M0+, comunicándose con ellos a través de buffers FIFO de cuatro palabras de 32 bits, que pueden encadenarse con el controlador DMA para transferencias de mayor tamaño.

El propósito fundamental del PIO es permitir la implementación de protocolos de comunicación digital personalizados y la generación de señales de temporización precisa sin consumir recursos del procesador principal (Everard, 2021). Esta capacidad resulta particularmente valiosa para aplicaciones que requieren protocolos no estándar, como el control de LEDs direccionables WS2812B, la generación de señales VGA o la emulación de buses de comunicación para periféricos antiguos. En esencia, el PIO proporciona una flexibilidad a nivel de hardware que pocos microcontroladores ofrecen, y constituye la principal ventaja diferencial del RP2040 frente a sus competidores.

Lenguajes de programación

Ambas plataformas ofrecen una amplia variedad de opciones en cuanto a lenguajes de programación, lo que las hace accesibles tanto a principiantes como a desarrolladores experimentados.

El ESP32 puede programarse en C y C++ utilizando el framework oficial ESP-IDF (Espressif IoT Development Framework), que proporciona acceso completo a todas las funcionalidades del SoC, incluyendo la gestión de Wi-Fi, Bluetooth, FreeRTOS y los periféricos. El ESP-IDF incluye un sistema de compilación basado en CMake y un conjunto de herramientas que facilitan la configuración del proyecto mediante la utilidad menuconfig. Adicionalmente, el ESP32 es compatible con el ecosistema Arduino, lo que permite programarlo en un subconjunto de C++ utilizando las bibliotecas y la sintaxis familiar para la comunidad Arduino. Esta compatibilidad reduce significativamente la curva de aprendizaje para quienes ya poseen experiencia con la plataforma Arduino. También es posible

programar el ESP32 en MicroPython, una implementación de Python 3 optimizada para microcontroladores, que ofrece un entorno interactivo y facilita la creación rápida de prototipos. A nivel más avanzado, la arquitectura Xtensa LX6 puede programarse en lenguaje ensamblador utilizando el conjunto de instrucciones de Xtensa, lo que permite optimizaciones de muy bajo nivel para secciones críticas de código.

La Raspberry Pi Pico ofrece un abanico similar de lenguajes. El kit de desarrollo de software oficial, el Pico SDK, está basado en C y proporciona acceso completo a todas las capacidades del RP2040, incluyendo la programación de las máquinas de estado PIO. Al igual que el ESP32, la Pico es compatible con el ecosistema Arduino a través del paquete de placas "Arduino Mbed OS RP2040", lo que permite utilizar el lenguaje Arduino (C++) y sus bibliotecas. La programación en MicroPython es particularmente popular en la Pico, gracias a la excelente integración con el entorno Thonny y a la disponibilidad de bibliotecas específicas para el RP2040. El soporte para MicroPython incluye acceso a las máquinas de estado PIO, lo que permite programarlas desde Python sin necesidad de escribir código en ensamblador PIO. Para desarrollos de muy bajo nivel, el RP2040 puede programarse en ensamblador ARM, aprovechando el conjunto de instrucciones Thumb-2 del Cortex-M0+, así como en el lenguaje ensamblador específico de las máquinas de estado PIO.

Entornos de desarrollo integrado (IDEs)

El desarrollo de aplicaciones para ambas plataformas se beneficia de una variedad de entornos de desarrollo integrado (IDEs) que abarcan desde opciones sencillas para principiantes hasta configuraciones profesionales altamente personalizables.

Para el ESP32, el IDE más accesible es el Arduino IDE (en sus versiones 1.8.x y 2.x), que tras instalar el soporte para placas ESP32 permite compilar y cargar programas con la misma facilidad que en una placa Arduino convencional. Sin embargo, para proyectos de mayor

envergadura, la combinación de Visual Studio Code con la extensión PlatformIO se ha convertido en el estándar de facto. PlatformIO es un ecosistema de desarrollo open-source que proporciona gestión de dependencias, compilación automatizada, depuración y soporte para más de 1500 placas, incluyendo toda la familia ESP32 (SunFounder, 2025). La integración de PlatformIO con VS Code permite trabajar con los frameworks Arduino y ESP-IDF en un mismo entorno, ofreciendo un sistema de compilación robusto que simplifica la gestión de proyectos complejos (Paduchuru, 2025). El ESP-IDF, por su parte, cuenta con su propio plugin para VS Code y Eclipse, así como con herramientas de línea de comandos para la configuración, compilación y depuración.

Para la Raspberry Pi Pico, el entorno Thonny es la opción recomendada para quienes se inician en MicroPython. Thonny es un IDE Python ligero que incluye soporte nativo para la Pico, permitiendo la conexión directa, la transferencia de archivos y la ejecución interactiva de código (SunFounder, 2024). Para desarrollos en C/C++, el Pico SDK puede utilizarse con VS Code, CLion, Eclipse y otras herramientas que admitan CMake. El SDK oficial proporciona ejemplos y plantillas que facilitan la configuración del entorno de compilación cruzada. PlatformIO también ofrece soporte para la Raspberry Pi Pico, lo que permite unificar el flujo de trabajo para quienes trabajan con ambas plataformas.

Ecosistema de desarrollo y sistemas operativos de tiempo real (RTOS)

El desarrollo para el ESP32 está íntimamente ligado a FreeRTOS. El marco de desarrollo oficial, ESP-IDF (Espressif IoT Development Framework), integra FreeRTOS de forma nativa, lo que significa que casi cualquier aplicación escrita para ESP32 es inherentemente multitarea. Espressif ha modificado FreeRTOS para soportar multiprocesamiento simétrico (SMP) en los dos núcleos del ESP32, permitiendo que las

tareas se distribuyan automáticamente entre los núcleos o se fijen a uno específico mediante afinidad de núcleo.

El ecosistema del RP2040 es más joven, pero está extremadamente bien estructurado. El SDK de C/C++ de Raspberry Pi Pico es elogiado por su claridad y falta de "magia" innecesaria, permitiendo a los desarrolladores experimentados tener un control total sobre el hardware. Para principiantes, la implementación de MicroPython y CircuitPython en la Raspberry Pi Pico es considerada la mejor de la industria, proporcionando una barrera de entrada mínima. Aunque el RP2040 no tiene un RTOS "horneado" de la misma manera que el ESP32, es totalmente compatible con FreeRTOS y muchos desarrolladores lo utilizan para gestionar la concurrencia entre sus dos núcleos Cortex-M0+.

Proyectos simulados y validación técnica

Sistema automático de control de temperatura con servo

Se analizará la simulación en la plataforma Wokwi de un termostato automatizado que lee un sensor DHT22 y actúa sobre un servomotor que simula el flujo de aire para mantener la temperatura y una pantalla LCD mostrando la información del sistema.

Proyecto con ESP32 (Arquitectura basada en RTOS)

La simulación utiliza un enfoque de multitarea simétrica (SMP). El sensor DHT22 se conecta al pin GPIO 4 y el servomotor al pin 18. El control se gestiona mediante dos tareas de FreeRTOS: tareaSensor para la adquisición de datos cada 2000ms y tareaControl para la lógica de decisión y actualización del LCD cada 500ms.

El uso de FreeRTOS permite que la latencia crítica de lectura del DHT22 no bloquee la actualización visual del LCD ni el movimiento del servo. La biblioteca ESP32Servo

aprovecha el hardware PWM específico del chip para garantizar señales estables de 50 Hz necesarias para el servomotor.

Link de la simulación: <https://wokwi.com/projects/463474921232147457>

Proyecto con Raspberry Pi Pico (Control de bajo nivel y temporización manual)

En la implementación de la Pico, el código utiliza una aproximación secuencial en el *loop()* principal. Se destaca la función *moverServo(int angulo)*, que implementa el protocolo PWM de forma manual mediante *digitalWrite* y *delayMicroseconds*.

A diferencia del ESP32, donde el hardware PWM gestiona el ciclo de trabajo de forma autónoma, la implementación analizada en la Pico requiere que la CPU mantenga el pulso activo durante 20ms por ciclo en un bucle for de 50 iteraciones para posicionar el servo. Esto demuestra la naturaleza determinística del RP2040, donde el programador tiene control total sobre los microsegundos de ejecución, aunque a costa de bloquear la ejecución de otras tareas durante el movimiento del actuador. El bus I2C se configura explícitamente en los pines GP4 (SDA) y GP5 (SCL) usando *Wire.setSDA* y *Wire.setSCL*.

Link de la simulación: <https://wokwi.com/projects/463478377967275009>

Conclusiones

Se concluye que el ESP32 constituye una plataforma superior para aplicaciones de alta carga transaccional y conectividad concurrente. Su arquitectura de doble núcleo, sumada a la integración nativa con un sistema operativo de tiempo real (FreeRTOS) mediante multiprocesamiento simétrico (SMP), permite una gestión de hilos de ejecución que minimiza la latencia en tareas de red sin comprometer la estabilidad de los periféricos.

El microcontrolador RP2040, a través de su subsistema de Entrada/Salida Programable (PIO), redefine la flexibilidad del hardware en sistemas embebidos de bajo costo. Esta arquitectura permite la ejecución de protocolos personalizados con una precisión temporal que el ESP32 no puede alcanzar de forma nativa, posicionando a la Raspberry Pi Pico como la opción predilecta para el control de precisión y la emulación de interfaces digitales complejas.

La Eficiencia Energética y Diseño de Alimentación en la Raspberry Pi Pico presenta una ventaja en proyectos alimentados por batería debido a su regulador buck-boost integrado (1.8V a 5.5V) y un consumo en modo activo significativamente menor (18mA frente a los 50-70mA del ESP32 sin radio activa).

Finalmente, se determina que la elección entre ambas plataformas no depende del rendimiento bruto, sino del ecosistema y la naturaleza del proyecto. El ESP32 es la solución óptima para el ecosistema IoT industrial y doméstico debido a su madurez en comunicaciones inalámbricas, mientras que el RP2040 se consolida como una herramienta educativa y de ingeniería de precisión sin precedentes en el mercado de microcontroladores ARM.

Referencias

Technical Too Much. (2025). *ESP32 vs Raspberry Pi Pico: ¿Cuál es mejor para proyectos*

DIY?. Website <https://technicaltoomuch.com/esp32-vs-raspberry-pi-pico/>

Olive, Z. (2025). *ESP32 vs. Pico 2: Comparando ESP32 con el RP2350 de Raspberry Pi.*

CoolPlayerDev. Website <https://coolplaydev.com/esp32-vs-pico-2>

TecnoloBlog (2024). *ESP32: Qué es, para qué sirve y cómo empezar a programarlo.*

Website. <https://www.tecnoloblog.com/que-es-esp32/>

Raspberry Pi Official WebSite. Microcontroller chips - Raspberry Pi Documentation [h](https://www.raspberrypi.com/documentation/microcontrollers/microcontroller-chips.html#microcontroller-overview)

<https://www.raspberrypi.com/documentation/microcontrollers/microcontroller-chips.html#microcontroller-overview>

ESPRESSIF Official WebSite. ESP32 – Documentation, WebSite

<https://www.espressif.com/en/products/socs/esp32>

Anexos

Tabla 1

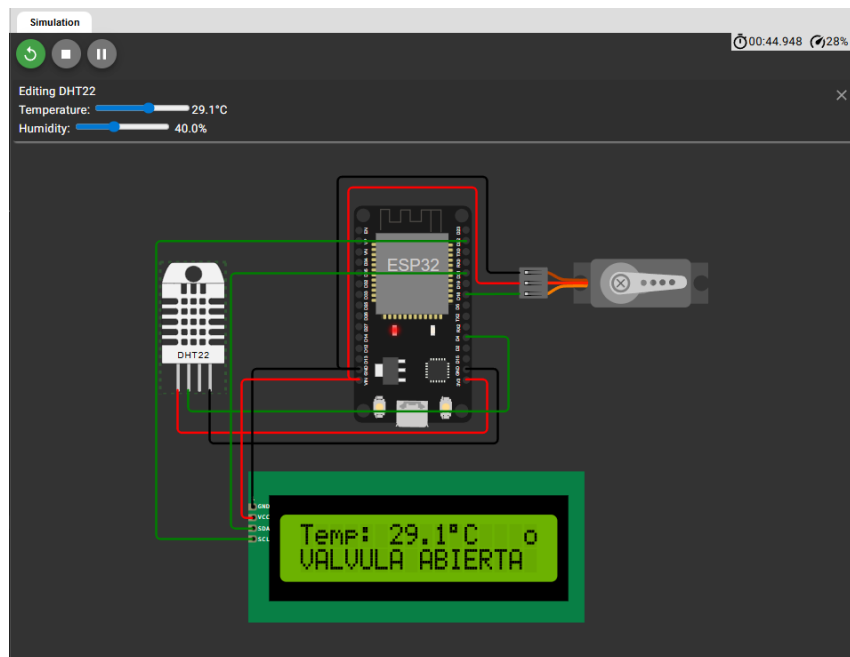
Tabla comparativa de tecnologías

Característica	ESP32	Raspberry Pi Pico (RP2040)
Arquitectura	Xtensa LX6 (32 bits)	ARM Cortex-M0+ (32 bits)
Núcleos	2	2
Frecuencia máxima	240 MHz	133 MHz (overclock hasta >400 MHz)
SRAM	520 KB	264 KB
ROM	448 KB	No (carga desde flash externa)
Flash	Externa (4–16 MB típica)	Externa en placa (2 MB)
Conectividad inalámbrica	Wi-Fi 802.11 b/g/n, Bluetooth 4.2/5.0 BLE	No integrada
GPIO	34	30 (26 en placa, 4 ADC)
ADC	12 bits, 18 canales	12 bits, 4 canales
DAC	2 × 8 bits	No
UART	3	2
SPI	4	2
I²C	2	2
PWM	16 canales	16 canales
PIO	No	8 máquinas de estado
USB	No nativo (disponible en S2/S3)	USB 1.1
RTOS	FreeRTOS (ESP-IDF)	No integrado (opcional)
Lenguajes	C, C++, MicroPython, Arduino, Xtensa ASM	C, C++, MicroPython, Arduino, ARM ASM, PIO ASM
Consumo (activo)	≈160 mA	≈90 mA
Sleep (deep)	10–20 μA	<50 μA

Nota. Datos recopilados de Espressif Systems (2024), Raspberry Pi Foundation (2021) y Supriyanto & Anggono (2025).

Figura 1

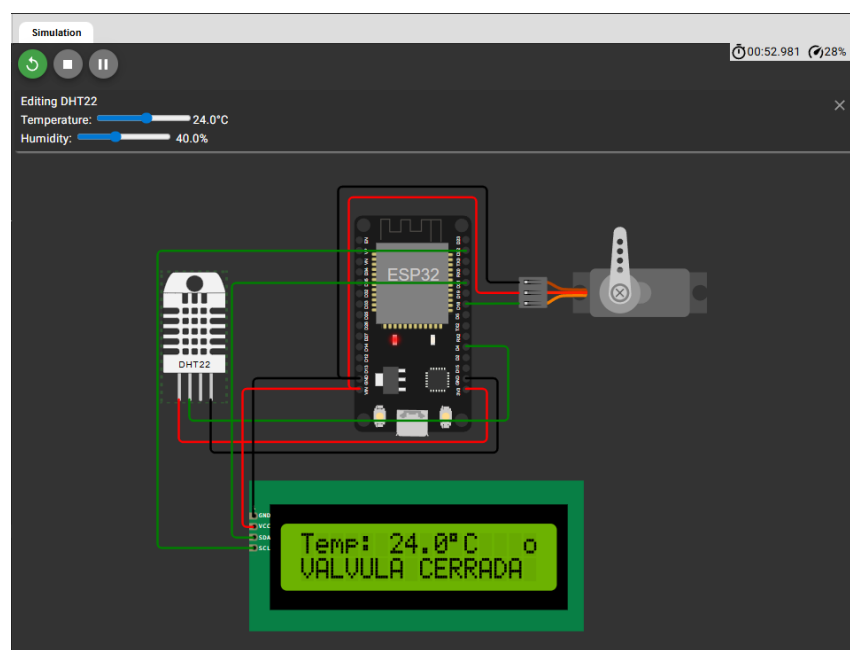
Simulación del sistema automático de control de temperatura con servo con ESP32



Nota: Simulación del DHT22 con una temperatura superior a 25°C por lo que el servo se acciona.

Figura 2

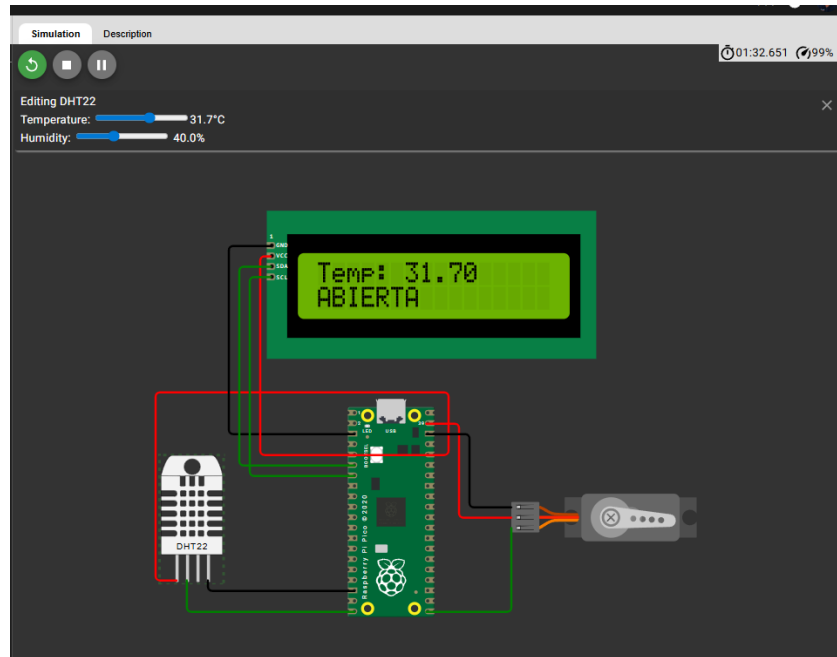
Simulación del sistema automático de control de temperatura con servo con ESP32



Nota: Simulación del DHT22 con una temperatura inferior a 25°C por lo que el servo se regresa a posición inicial.

Figura 3

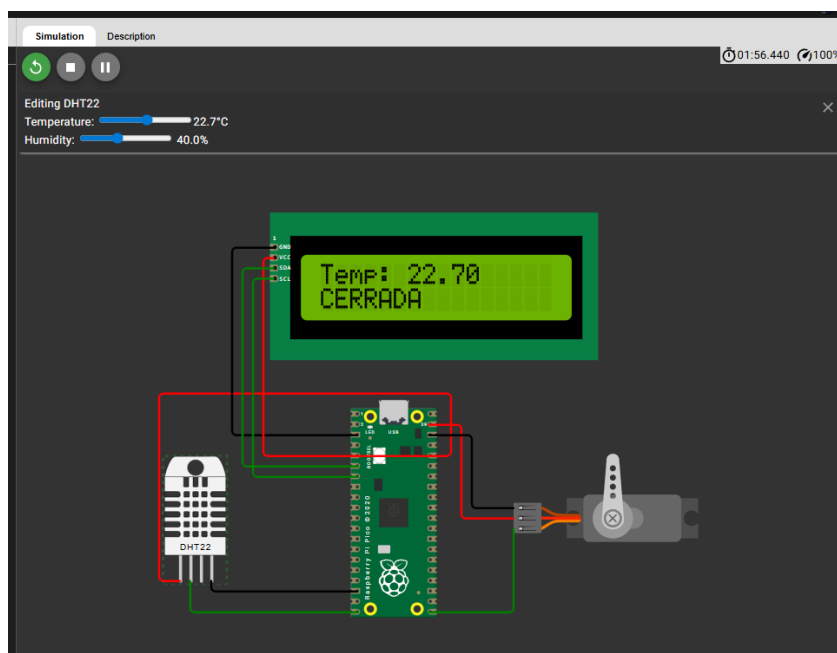
Simulación del sistema automático de control de temperatura con servo con Raspberry Pi



Nota: Simulación del DHT22 con una temperatura superior a 25°C por lo que el servo se acciona.

Figura 4

Simulación del sistema automático de control de temperatura con servo con Raspberry Pi



Nota: Simulación del DHT22 con una temperatura superior a 25°C por lo que el servo se regresa a su posición inicial.

Código implementado en simulaciones

Código del ESP32

```
#include <Arduino.h>
#include <DHT.h>
#include <ESP32Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define DHTPIN 4
#define DHTTYPE DHT22
#define SERVO_PIN 18

DHT dht(DHTPIN, DHTTYPE);
Servo servo;
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Variable compartida
float temperatura = 0;

// Control de cambios
float tempAnterior = -100;
bool estadoAnterior = false;

// ----- TAREA SENSOR -----
void tareaSensor(void *pvParameters) {
    while (true) {
        float t = dht.readTemperature();

        if (!isnan(t)) {
            temperatura = t;
            Serial.print("Temp leida: ");
            Serial.println(temperatura);
        }

        vTaskDelay(2000 / portTICK_PERIOD_MS);
    }
}

// ----- TAREA CONTROL -----
void tareaControl(void *pvParameters) {
    while (true) {

        bool estadoActual = (temperatura > 25);

        // Solo actualizar si cambia temperatura o estado
        if (temperatura != tempAnterior || estadoActual != estadoAnterior) {

            // Línea 1: temperatura
```

```
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");

    lcd.print(temperatura, 1); // 1 decimal
    lcd.print((char)223);      // símbolo °
    lcd.print("C ");          // limpiar residuos

    // Línea 2: estado
    lcd.setCursor(0, 1);

    if (estadoActual) {
        servo.write(90);
        lcd.print("VALVULA ABIERTA ");
    } else {
        servo.write(0);
        lcd.print("VALVULA CERRADA ");
    }

    // Guardar estados
    tempAnterior = temperatura;
    estadoAnterior = estadoActual;
}

vTaskDelay(500 / portTICK_PERIOD_MS);
}
}

// ----- SETUP -----
void setup() {
    Serial.begin(115200);

    dht.begin();

    servo.attach(SERVO_PIN);

    lcd.init();
    lcd.backlight();

    // Mensaje inicial
    lcd.setCursor(0, 0);
    lcd.print("Sistema iniciado");
    lcd.setCursor(0, 1);
    lcd.print("Esperando datos");
    delay(2000);

    // Crear tareas
    xTaskCreate(tareaSensor, "Sensor", 2048, NULL, 1, NULL);
    xTaskCreate(tareaControl, "Control", 2048, NULL, 1, NULL);
}
```

```
// ----- LOOP -----  
void loop() {  
  // No se usa en RTOS  
}
```

Código del RaspBerry Pi Pico

```
#include <Arduino.h>  
#include <DHT.h>  
#include <Wire.h>  
#include <LiquidCrystal_PCF8574.h>  
  
#define DHTPIN 15  
#define DHTTYPE DHT22  
#define SERVO_PIN 16  
  
DHT dht(DHTPIN, DHTTYPE);  
LiquidCrystal_PCF8574 lcd(0x27); // cambiar a 0x3F si necesario  
  
void moverServo(int angulo) {  
  int pulso = map(angulo, 0, 180, 500, 2500);  
  
  for (int i = 0; i < 50; i++) {  
    digitalWrite(SERVO_PIN, HIGH);  
    delayMicroseconds(pulso);  
    digitalWrite(SERVO_PIN, LOW);  
    delay(20 - pulso / 1000);  
  }  
}  
  
void setup() {  
  Serial.begin(115200);  
  delay(2000);  
  
  dht.begin();  
  pinMode(SERVO_PIN, OUTPUT);  
  
  Wire.setSDA(4);  
  Wire.setSCL(5);  
  Wire.begin();  
  
  lcd.begin(16, 2);  
  lcd.setBacklight(255);  
  
  lcd.setCursor(0, 0);  
  lcd.print("Sistema OK");  
  delay(2000);  
}
```

```
void loop() {  
  float t = dht.readTemperature();  
  
  lcd.setCursor(0, 0);  
  
  if (!isnan(t)) {  
    Serial.println(t);  
  
    lcd.print("Temp: ");  
    lcd.print(t);  
    lcd.print(" ");  
  
    lcd.setCursor(0, 1);  
  
    if (t > 25) {  
      moverServo(90);  
      lcd.print("ABIERTA ");  
    } else {  
      moverServo(0);  
      lcd.print("CERRADA ");  
    }  
  
    } else {  
      Serial.println("Error DHT");  
      lcd.print("Error sensor ");  
    }  
  
  delay(2000);  
}
```